Aula 29 - Autenticação JWT com WebFlux (Backend) + Vue.js (Frontend)

Duração: 4 horas (2h backend + 2h frontend)

Parte 1: Backend com Spring WebFlux (2 horas)

1. Configuração do Projeto

Dependências (`pom.xml`)

xml

```xml
<!-- Spring Security Reactive -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<!-- JWT -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>
```

2. Implementação do JWT

Classe de Utilidades (`JwtUtil.java`)

java

```java
@Component
public class JwtUtil {
    private final String SECRET_KEY = "chave-secreta-super-forte-123456";

    public Mono<String> generateToken(Authentication auth) {
        return Mono.just(Jwts.builder()
            .setSubject(auth.getName())
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 86400000)) // 24h
            .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
            .compact());
    }
```

```
    public Mono<Claims> validateToken(String token) {
        return Mono.just(Jwts.parser()
            .setSigningKey(SECRET_KEY)
            .parseClaimsJws(token)
            .getBody());
    }
}
```

Filtro de Autenticação (`JwtAuthFilter.java`)

java

```
public class JwtAuthFilter implements WebFilter {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, WebFilterChain chain) {
        String token = extractToken(exchange.getRequest());
        if (token == null) return chain.filter(exchange);

        return jwtUtil.validateToken(token)
            .flatMap(claims -> {
                Authentication auth = new UsernamePasswordAuthenticationToken(
                    claims.getSubject(), null, List.of(new SimpleGrantedAuthority("ROLE_U
SER"))
                );
                return chain.filter(exchange)
                    .contextWrite(ReactiveSecurityContextHolder.withAuthentication(aut
h));
            });
    }
}
```

3. Endpoints de Autenticação

Controller (`AuthController.java`)

java

```
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {
    private final ReactiveAuthenticationManager authManager;
    private final JwtUtil jwtUtil;

    @PostMapping("/login")
    public Mono<ResponseEntity<Map<String, String>>> login(@RequestBody AuthRequest reque
st) {
        return authManager.authenticate(
            new UsernamePasswordAuthenticationToken(request.username(), request.password
```

```
())
        ).flatMap(auth -> jwtUtil.generateToken(auth)
          .map(token -> ResponseEntity.ok(Map.of("token", token)));
    }
}
```

Parte 2: Frontend com Vue.js (2 horas)

1. Serviço de Autenticação (`auth.service.js`)

javascript

```
import axios from 'axios';
import { ref } from 'vue';

const API_URL = 'http://localhost:8080/api/auth';
const token = ref(localStorage.getItem('token'));

export function useAuth() {
    const login = async (username, password) => {
        try {
            const response = await axios.post(`${API_URL}/login`, { username, password
});
            token.value = response.data.token;
            localStorage.setItem('token', token.value);
            return true;
        } catch (error) {
            console.error("Login failed:", error);
            return false;
        }
    };

    return { token, login };
}
```

2. Interceptor para JWT (`http.js`)

javascript

```
axios.interceptors.request.use(config => {
    const token = localStorage.getItem('token');
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
});

axios.interceptors.response.use(
```

```
    response => response,
    error => {
        if (error.response.status === 401) {
            window.location.href = '/login';
        }
        return Promise.reject(error);
    }
);
```

3. Componente de Login (`Login.vue`)

vue

```
<template>
    <form @submit.prevent="handleLogin">
        <input v-model="username" placeholder="Usuário">
        <input v-model="password" type="password" placeholder="Senha">
        <button type="submit">Entrar</button>
    </form>
</template>

<script setup>
import { useAuth } from '@/services/auth.service';

const { login } = useAuth();
const username = ref('');
const password = ref('');

const handleLogin = async () => {
    if (await login(username.value, password.value)) {
        router.push('/');
    }
};
</script>
```

4. Roteamento Protegido (`router.js`)

javascript

```
import { createRouter } from 'vue-router';
import { useAuth } from './services/auth.service';

const { token } = useAuth();

const router = createRouter({
    routes: [
        { path: '/', component: Home, meta: { requiresAuth: true } },
        { path: '/login', component: Login }
    ]
```

```
});

router.beforeEach((to) => {
    if (to.meta.requiresAuth && !token.value) {
        return '/login';
    }
});
```

Tarefa para Casa

1. **Implementar Logout:** Limpar token do localStorage e redirecionar para `/login`.
2. **Refresh Token:** Adicionar renovação automática do token antes da expiração.

Próxima Aula

- **Tópico:** WebSockets com WebFlux + Vue.js.
- **Atividade prática:** Chat em tempo real.

Resultado Esperado

- **Backend:**
  - Rota `/api/auth/login` gerando tokens JWT.
  - Rotas protegidas por `JwtAuthFilter`.
- **Frontend:**
  - Login persiste token no localStorage.
  - Requests autenticadas automaticamente.

plaintext

```
✅ Autenticação JWT funcional
✅ Integração frontend-backend
✅ Pronto para WebSockets
```

https://miro.medium.com/v2/resize:fit:1400/1*HXGZhRkG0Lf8YIrGJ1Hl4A.png *(Fluxo JWT entre Vue.js e WebFlux)* 🚀